



Introduction to Node.js

Outline



➤ **Part-1: Introduction to Node.js**

- What is Node.js?
- Why Node.js?
- Use of Node
- Node Installation
- Reading/Writing Files

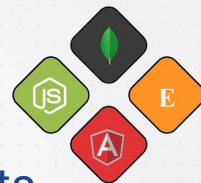
➤ **Part-2: Synchronous vs Asynchronous**

- Difference
- Reading/Writing File Asynchronously
- Creating Server
- Routing
- Build API

➤ **Part-3: Building HTML Template**

- Build HTML template
- Filling HTML template with dynamic data
- Create own module

Node.js?



- ❑ V8 is an open source JavaScript engine developed by Google. Its written in C++ and is used in Google Chrome Browser.
- ❑ Node.js runs on V8. It is open source.
- ❑ It is used for creating server-side web applications. It is not a framework.
- ❑ Node.js uses an event-driven, non-blocking I/O model, which makes it lightweight.
- ❑ It makes use of event-loops via JavaScript callback functionality to implement the non-blocking I/O.
- ❑ Programs for Node.js are written in JavaScript but not in the same JavaScript we are use to. There is no DOM implementation provided by Node.js.
- ❑ Everything inside Node.js runs in a single-thread.

Why Node.js?



- ❑ Single-threaded, based on event driven, non-blocking I/O model.
- ❑ Perfect for building fast and scalable data-intensive apps.
- ❑ Companies like NETFLIX, UBER, PayPal, ebay are using Node.
- ❑ **JavaScript across the entire stack:** faster and more efficient development.
- ❑ **NPM** (Node Package Manager) huge library of open-source packages available for everyone for free.
- ❑ **Very active** developer community.
- ❑ It is easy to learn for beginners as it uses JavaScript code.

Use of Node.js



- ☐ API with database behind it (preferably NoSQL).
- ☐ Data streaming (think YouTube)
- ☐ Real-time chat application.
- ☐ Server-side web application.
- ☐ **Do not use Node for applications with heavy server-side processing (CPU-intensive)**

Node Installation



- ❑ Install Node.js from official website <https://nodejs.org/en/>
- ❑ From command prompt you can use the following commands
 - **node -v** → to check the version of node.
 - **npm -version** → to check that npm is installed or not.
 - **Node** → to start node command prompt.
 - Here you can use JavaScript same as we use on browser.
 - Use **ctrl + c** two times to exit from node command prompt.



Running JavaScript Outside Browser



- ☐ Open [resource](#) in vs code.
- ☐ Open the terminal in vs code any use command **node**.
- ☐ Now you can write and run JavaScript code in terminal.
const name='Mohit'
- ☐ Press two times tab to see the help.
- ☐ To see all the methods of a particular class use ***Class_Name. Tab.***
- ☐ **Example:** String.Tab
- ☐ Create a JavaScript file named **index.js**. Write some JavaScript code in this file.
- ☐ Now to run this file use command **node index.js**

Node in Built Modules



- Node.js provides several built-in modules that are pre-installed and ready to use in your application without the need for any additional installation.
- Some of them are as follow:
 - ***fs (File System)***
 - ***buffer***
 - ***http***
 - ***path***
 - ***os***
 - ***util***
 - ***url***

fs Module



- Used for interacting with the file system (reading, writing, updating, and deleting files).
- Key Methods:
 - **fs.readFile(path, callback)** - Reads a file asynchronously.
 - **fs.writeFile(path, data, callback)** - Writes data to a file.
 - Synchronous versions like fs.readFileSync, fs.writeFileSync.
 - **fs.appendFile(path, data, callback)** - Appends data to a file.
 - **fs.unlink(path, callback)** - Deletes a file.
 - **fs.mkdir(path, callback)** - Creates a directory.

Reading/Writing Files in Node



- First import the fs module inside your code.
- Use `readFileSync` function to read the file.
- Use `writeFileSync` function to write the file.

```
const fs = require("fs");  
  
const inputText = fs.readFileSync("./txt/input.txt", "utf-8");  
const outText = `This is all about avocado: ${inputText}. \n Created on  
${Date.now()} `;  
  
fs.writeFileSync("./txt/output.txt", outText);  
console.log(outText);
```

- Now run the file using command ***node index.js***

Synchronous vs Asynchronous



```
const fs = require("fs");
const data = fs.readFileSync("./txt/input.txt", "utf-8");
console.log(data);
console.log("Reading the file.....");
```



SYNCHRONOUS



BLOCKING

NON-BLOCKING



ASYNCHRONOUS



```
const fs = require("fs");
fs.readFile("./txt/input.txt", "utf-8", (err, data) => {
  console.log(data);
});
console.log("Reading the file.....");
```

Reading/Writing File Asynchronously



□ Resources

```
const fs = require("fs");
fs.readFile("./txt/start.txt", "utf-8", (err, data) => {
  fs.readFile(`./txt/${data}.txt`, "utf-8", (err, data1) => {
    fs.readFile("./txt/input.txt", "utf-8", (err, data2) => {
      fs.writeFile("./txt/final.txt", `${data2}\n${data1}`, "utf-8", (err) => {
        console.log("Data file created.....");
      });
    });
  });
});
console.log("File reading and writing in progress....");
```

buffer Module



- ❑ Used to handle binary data in Node.js, especially for streams.
- ❑ Key Features:
 - **Create a buffer:** `Buffer.from(array)` or `Buffer.alloc(size)`.
 - **Convert buffer to string:** `buffer.toString('utf8')`.
 - **Combine buffers:** `Buffer.concat([buf1, buf2])`.
 - **Length of a buffer:** `buffer.length`.

http Module



- Used for building HTTP servers and making HTTP requests.
 - **Create a server:** Use `http.createServer()` to create a server.
 - **Make a Request:** Use `http.get()` to make a request.

Creating Simple Web Server



- ❑ Here we will create a simple web server that will accept a simple request and send the response.
- ❑ For this we will use ***http*** module.
- ❑ While creating the server we will do:
 - Create the server
 - Start the server
- ❑ To create the server use ***http.createServer()*** method. This method takes a callback function as argument.
- ❑ To start the server use ***server.listen(port, ip, callback)*** method. This method take port, ip and a callback function as argument.

Creating Simple Web Server



```
const http = require("http");

// SERVER
const server = http.createServer((req, res) => {
  res.end("Hello from the server");
});

server.listen(8080, "127.0.0.1", () => {
  console.log("Listening started at port 8080");
});
```

If we print the *req* then it will print the complete request object

This is the ip address for the localhost. We can also use localhost in place of this ip.

Routing in Node



- Routing refers to determining how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, and so on).
- Each route can have one or more handler functions, which are executed when the route is matched.
- Routing is implementing different actions of different URLs.
- In real world projects routing can be very complex. So in real world we use **EXPRESS**.
- In node for doing routing first we analyze the **URL**. For this we will use **url** module.

Routing in Node



```
const http = require("http");
const url = require("url");
// SERVER
const server = http.createServer((req, res) => {
  const pathName = req.url;
  if (pathName === "/" || pathName === "/home") {
    res.end("This is the HOME page");
  } else if (pathName === "/contact") {
    res.end("This is the CONTACT page");
  } else {
    res.writeHead(404, {
      "content-type": "text/html",
      "my-custom-head": "costom-head",
    });
    res.end("<h1>Page not found!</h1>");
  }
});
server.listen(8080, "localhost", () => {
  console.log("Listening started at port 8080");
});
```

It will return the requested URL

We can set the response header like response code, content type

JSON Introduction



- JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy to read, write, and parse.
- It is widely used in Node.js for communication between servers and clients, configuration, data storage, and APIs.
- **Following are the characteristics of JSON:**
 - Text-Based Format
 - Language-Independent
 - Lightweight
 - Data Types Supported: Strings, Numbers, Booleans, Null, Arrays, Objects.



JSON in Node js



- In Node.js, JSON is a built-in global object used to handle JSON data.
- It includes two main methods:
 - **JSON.parse():** Converts a JSON string into a JavaScript object.

```
const jsonstring = '{"name":"student-1", "age":20,  
"branch":"CSE"}';
```

```
const obj = JSON.parse(jsonstring);  
console.log(obj);
```

- **JSON.stringify():** Converts a JavaScript object into a JSON string.

```
const jsobj = {name: "student-1", age: 20,  
  branch: "CSE"  
};  
console.log(JSON.stringify(jsobj));
```

Reading JSON from File in Node js



- To read the JSON from a file in Node js, we use `readFile()` method of **fs** module.
- After reading the data, we will convert this data to JavaScript object.

```
const fs = require('fs');  
fs.readFile('files/data.json', 'utf-8', (err, data) => {  
  const jsobj = JSON.parse(data);  
  console.log(jsobj);  
})
```

Writing JSON to File in Node js



- To write the JSON to file in Node js, we use `writeFile()` method of **fs** module.
- Before writing JSON to file, first we have to convert JavaScript object to string.

```
const fs = require('fs');
const data = {
  name: 'student one',
  age: 21,
  branch: 'CSE'
};
const objstr = JSON.stringify(data);
fs.writeFile('files/student.json', objstr, () => {
  console.log('Data written');
});
```

How JSON is Used in Node.js?



- ❑ **Handling API Data:** JSON is the standard format for exchanging data in RESTful APIs. In Node.js, we can send and receive JSON data in HTTP requests and responses.
- ❑ **Reading/Writing Files:** JSON is used to store and retrieve data in configuration files, such as .json files.
- ❑ **Configuration Management:** Node.js applications often use JSON files for environment configurations, like package.json for managing dependencies.
- ❑ **Database Communication:** Many NoSQL databases like MongoDB use JSON-like documents to store data.

req & res Objects in Node js



- In Node.js, the **req** (request) and **res** (response) **objects** are part of the HTTP module and are used to handle incoming requests and send responses in a server.

Request (req) Object



- The **req** object represents the HTTP request and contains details about the client's request.
- Following are common **req** properties:
 - **req.url**: The URL of the request.
 - **req.method**: The HTTP method used (e.g., GET, POST).
 - **req.headers**: An object containing the request headers.
 - **req.query**: Contains query string parameters (requires Express).
 - **req.params**: Contains route parameters (requires Express).
 - **req.body**: Contains the body of the request (requires middleware like `express.json()` for parsing JSON).
 - **req.cookies**: Contains cookies sent by the client (requires `cookie-parser` middleware).

Response (res) Object



- The **res** object represents the HTTP response that the server sends back to the client.
- Following are common **res** Methods:
 - **res.send()**: Sends a response to the client. It can send a string, a JSON object, or a buffer.
 - **res.json()**: Sends a JSON response to the client. It automatically sets the Content-Type header to application/json.
 - **res.status()**: Sets the HTTP status code for the response.
 - **res.end()**: Ends the response without sending any data.
 - **res.redirect()**: Redirects the client to a different URL.
 - **res.set()**: Sets custom headers for the response.
 - **res.cookie()**: Sets a cookie in the response (requires cookie-parser middleware).
 - **res.type()**: Sets the Content-Type header explicitly.
 - **res.download()**: Sends a file for download.

Callback Function



- A callback function in Node.js is a function that is passed as an argument to another function and is executed after the completion of that function.
- It is a fundamental concept in asynchronous programming in Node.js.
- Following are the characteristics of Callback Functions:
 - **Asynchronous Execution:**
 - Callbacks are used to handle tasks that take time to complete, such as reading files, making HTTP requests, or interacting with databases.
 - Instead of blocking the program while waiting for the task to complete, the callback is executed when the task finishes.
 - **Error Handling:**
 - Callback functions in Node.js often follow the pattern `callback(error, result)`, where the first parameter is for errors and the second is for results.

Asynchronous Programming



- Asynchronous programming is a programming paradigm that allows code to run without blocking the execution of other tasks.
- It is useful in handling tasks that take time to complete, such as file I/O, database queries, or API calls, while maintaining the responsiveness of the application.

Key Concepts of Asynchronous Programming



❑ **Non-blocking Execution:**

- Code does not block while waiting for a task to finish.
- The program can perform other operations while waiting for the task to complete.

❑ **Event Loop:**

- In Node.js, the event loop is the mechanism that handles asynchronous operations by listening for events and dispatching their corresponding callbacks.
- It is responsible for executing the code, collecting & processing events and executing queued sub task.
- The event loop pushes the tasks from the task queue to the call stack.

❑ **Callback Functions**

❑ **Promises**

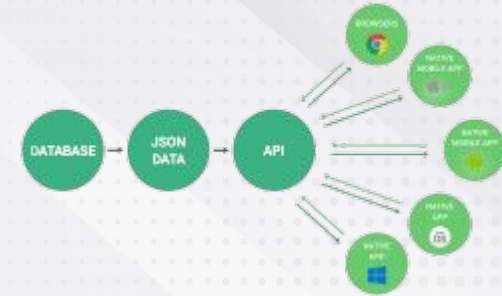
❑ **Async/Await**

What is API?

- Application Programming Interface: A piece of software that can be used by another piece of software, in order to allow applications to talk to each other.
- Node.js fs or http APIs.
- Browser's DOM JavaScript API.
- With object-oriented programming, when exposing methods to the public, we are creating an API.



👉 Web APIs



What is REST API?



- A REST API (Representational State Transfer Application Programming Interface) in Node.js is a server-side application that provides a structured way to expose resources (like users, products, or orders) over HTTP.
- It follows the REST architectural principles and allows clients (web apps, mobile apps, or other systems) to perform CRUD (Create, Read, Update, Delete) operations on these resources.

The REST Architecture



- ☐ Separate API into logical resources.
- ☐ Expose structured, resource-based URLs.
- ☐ Use HTTP methods
- ☐ Send data as JSON
- ☐ Be stateless.

Separate API into Logical Resources



- **Resource:** Object or representation of something, which has data associated to it.
- Any information that can be named can be resource.

students

users

courses

URL

<https://www.mycollege.com/addNewStudent>

ENDPOINT

/getStudents

/updateStudent

/updateStudentByBranch

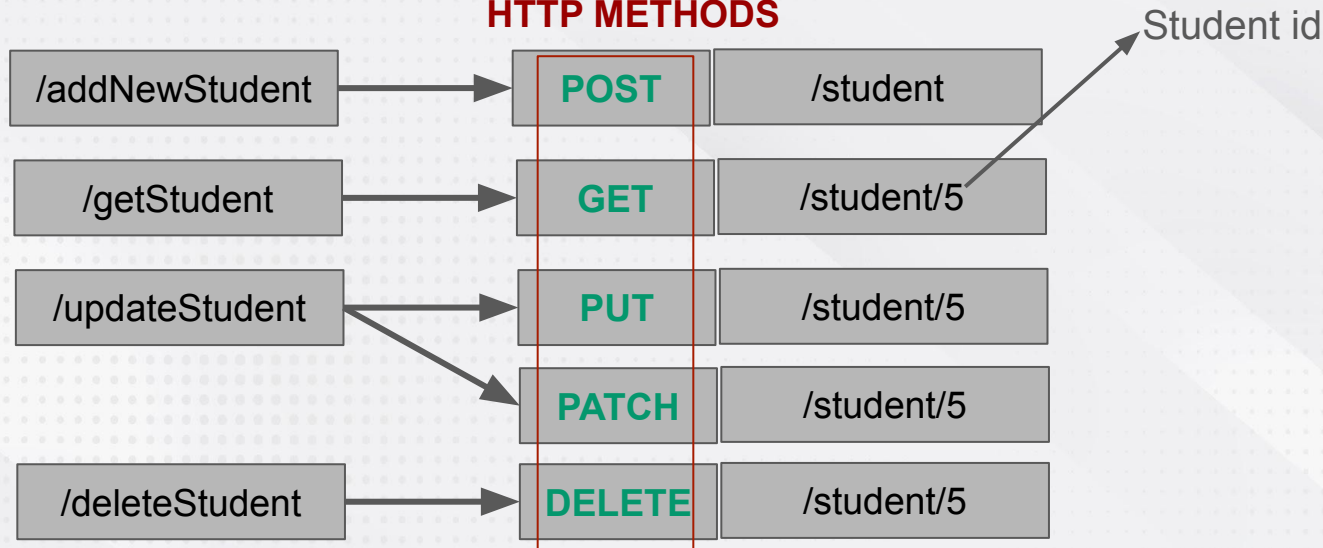
/deleteUser

👉 Endpoints should contain only resources and use HTTP methods for actions

Expose resource-based URLs, use HTTP Methods



HTTP METHODS



👉 Create

👉 Read

👉 Update

👉 Delete

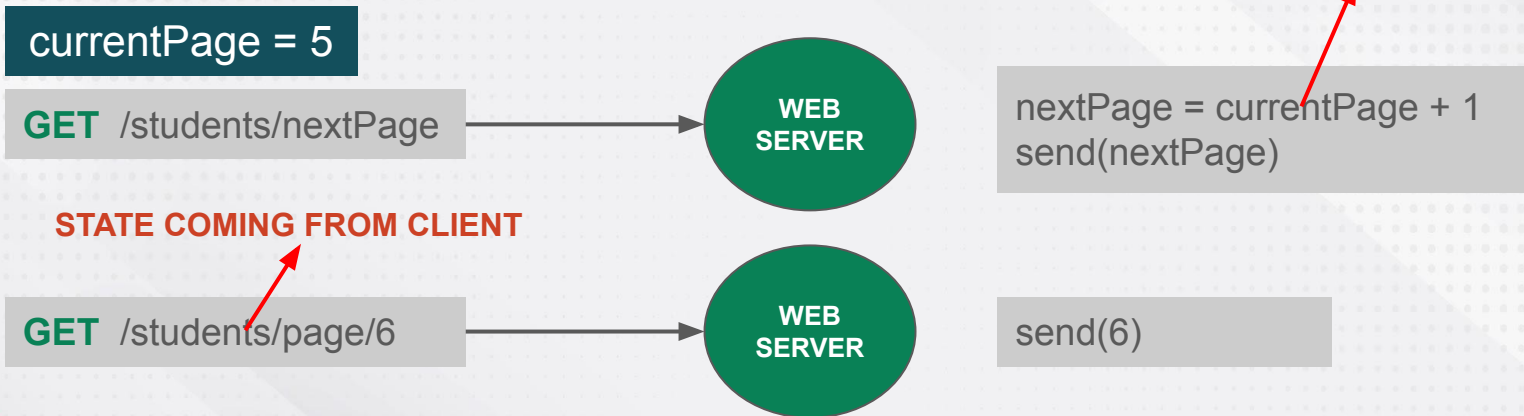
CRUD OPERATIONS



Be Stateless



- **Stateless RESTful API:** All states is handled on the client. This means that each request must contain all the information necessary to process a certain request.
- The server should not have to remember previous requests.
- Examples of state: `loggedIn`, `currentPage`



Build Simple API Using Node



- What is an API?
 - An API is a service from which we can request some data.
- For better understanding use this resource. [resource](#)
- In this example we are going to read the data from a JSON file and send it to the client.

```
const http = require("http");
const url = require("url");
const fs = require("fs");
// SERVER
const server = http.createServer((req, res) => {
  const pathName = req.url;
  if (pathName === "/api") {
    fs.readFile("./dev-data/dat2a.json", "utf-8", (err, data) => {
      const productData = JSON.parse(data);
      console.log(data);
      res.end("API data");
    });
  } else {
    res.end("Page not found!");
  }
});
server.listen(8080, "localhost", () => {
  console.log("Listening started at port 8080");
});
```

Build Simple API Using Node



- While reading the file (./) . denote the directory from which we run the node command.
- Here in place of . we can use the predefined variable **`__dirname`**

```
const http = require("http");
const url = require("url");
const fs = require("fs");
// SERVER
const server = http.createServer((req, res) => {
  const pathName = req.url;
  if (pathName === "/api") {
    fs.readFile(`_${__dirname}/dev-data/data.json`,
      "utf-8", (err, data) => {
        const productData = JSON.parse(data);
        res.writeHead(200, { "content-type":
          "application/json" });
        res.end(data);
      });
  } else {
    res.end("Page not found!");
  }
});
server.listen(8080, "localhost", () => {
  console.log("Listening started at port 8080");
});
```

Build Simple API Using Node



- The problem with this code is that when the user hits the */api* route, the file will be read every time.
- To overcome this problem we can read the file synchronously.
- In this case the file will be read only once.

```
const http = require("http");
const url = require("url");
const fs = require("fs");
// SERVER
const data =
  fs.readFileSync(`${__dirname}/dev-data/data.json`,
    "utf-8");
const server = http.createServer((req, res) => {
  const pathName = req.url;
  if (pathName === "/api") {
    res.writeHead(200, { "content-type":
      "application/json" });
    res.end(data);
  } else {
    res.end("Page not found!");
  }
});
server.listen(8080, "localhost", () => {
  console.log("Listening started at port 8080");
});
```

What is Promise?



- A Promise in Node.js is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.
- Promises make it easier to work with asynchronous code, replacing the traditional callback-based approach and improving code readability and error handling.

Features of Promise



- ❑ **State Management:**
 - **Pending:** The initial state, neither fulfilled nor rejected.
 - **Fulfilled:** The operation completed successfully, and a result is available.
 - **Rejected:** The operation failed, and an error is available.
- ❑ **Chaining:** Promises can be chained using **.then()** for sequential execution of asynchronous tasks.
- ❑ **Error Handling:** Errors are handled in a centralized way using **.catch()**.
- ❑ **Non-blocking:** Promises allow asynchronous operations to run without blocking the main thread.

How Promise Works?



- A Promise takes a function (executor) as an argument. This function has two parameters:
 - **resolve:** Call this function when the operation completes successfully.
 - **reject:** Call this function when the operation fails.

Create Promise in Node js



```
const http = require('http');
const promise = new Promise((resolve, reject) => {
  const success = true;
  if (success) {
    resolve('promise is resolved');
  } else {
    reject('promise is rejected');
  }
})
const server = http.createServer((req, res) => {
  promise.then(result => {
    res.end(result);
  }).catch(error => {
    res.end(error);
  })
});

server.listen(8080, '127.0.0.1', () => {
  console.log('server started....');
})
```

Reading file using Promise in Node js



```
const http = require('http');
const fs = require('fs').promises;
const server = http.createServer((req, res) => {
  fs.readFile('files/data.json', 'utf-8').then(result => {
    res.end(result);
  }).catch(error => {
    res.end(error);
  })
});
server.listen(8080, '127.0.0.1', () => {
  console.log('server started....');
});
```

What is Template Engine in Node js?



- A template engine in Node.js is a tool that allows developers to generate dynamic HTML pages by combining a predefined HTML structure with dynamic data.
- It provides a way to embed server-side data and logic into HTML templates, making it easier to create dynamic and reusable web pages.

Features of Template Engine



- ❑ **Dynamic Content Rendering:** Inserts variables, loops, and conditions into HTML to render dynamic content.
- ❑ **Separation of Concerns:** Separates HTML structure (view) from application logic (controller).
- ❑ **Reusability:** Allows creating reusable components, such as headers, footers, and navigation menus.
- ❑ **Ease of Use:** Simplifies the process of generating HTML pages dynamically without writing raw HTML strings in the code.

How Template Engine Work?



- ☐ The developer creates a template with placeholders (variables, loops, or conditions).
- ☐ Data is passed from the server (Node.js) to the template.
- ☐ The template engine processes the template, replacing placeholders with the actual data and logic results.
- ☐ The final HTML page is sent to the client.

EJS(Embedded JavaScript)



- EJS is a simple and popular templating engine for Node.js that enables you to generate dynamic HTML content by embedding JavaScript code directly within your templates.
- It combines plain HTML with JavaScript to allow seamless integration of server-side data into web pages.
- We can install EJS by using following command:
npm install ejs
- We can use JavaScript variable in ejs as:
<%= variable %>
- We can use if condition in ejs as:
<% if (condition) { %>
<p>Condition is true</p>
<% } else { %>
<p>Condition is false</p>
<% } %>

EJS(Embedded JavaScript) contd..



- We can use for loop in ejs as:

```
<ul>  
  <% for (let i = 0; i < items.length; i++) { %>  
    <li><%= items[i] %></li>  
  <% } %>  
</ul>
```

Steps for Creating App with EJS



- Install `ejs` into your project by using ***npm install*** ***ejs*** command.
- Create a JavaScript file(Node js file) that will create and run server. This will also read the data from JSON file.

```
const http = require('http');
const url = require('url');
const fs = require('fs');
const ejs = require('ejs');

const data = fs.readFileSync('files/data.json');
const dataobj = JSON.parse(data);
const server = http.createServer((req, res) => {
  const endpoint = req.url;
  if (endpoint === '/home') {
    ejs.renderFile('views/index.ejs', { dataobj }, (err,
html) => {
      res.end(html);
    })
  }
})
server.listen(8080, '127.0.0.1', () => {
  console.log('server started');
})
```

Steps for Creating App with EJS



- Create a views folder in your project.
- In this folder create one ejs file named ***index.ejs*** will following code.

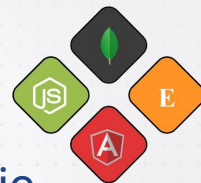
```
<body>
  <table id="customers">
    <tr>
      <th>Id</th>
      <th>Name</th>
      <th>Neutrients</th>
      <th>Image</th>
    </tr>
    <% for (let i=0; i<dataobj.length; i++) { %>
      <tr>
        <td><%= dataobj[i].id %></td>
        <td><%= dataobj[i].productName %></td>
        <td><%= dataobj[i].nutrients %></td>
        <td><%= dataobj[i].image %></td>
      </tr>
    <%}%>
  </table>
</body>
```

Building HTML Template



- ❑ For this we are going to use the static HTML template. [Static template](#)
- ❑ Here we have two pages: one is **overview** and another is **product**.
- ❑ Now we want to fill the content of these two pages from the JSON file.
- ❑ Now in the **product** page replace the data that we want to add dynamically with a **placeholder**.
- ❑ We can take any name for the placeholder. Like `{ %PRODUCTNAME% }`
- ❑ Then through node code we will replace the placeholder with the real data.
- ❑ Change the name of file from **product.html** to **template-product.html**.

Building HTML Template



- ☐ Now change the static content of overview.html page with dynamic one.
- ☐ For this remove all the cards from the overview page because we do not know the number of cards in advance.
- ☐ Put the placeholder inside the **card-container** div.
- ☐ Create new file named **template-overview.html** which will contain one card.
- ☐ Now the templates will look like [template](#).

Filling HTML Template with Dynamic Content



- ❑ Inside the node js file create routes for **product** and **overview** page.
- ❑ Read all the create html templates synchronously in the index.js file and store them in a variable.
- ❑ Read the JSON file and create a JSON object of this file.
- ❑ This dataObj is an object array. Now loop over this object and foreach object replace the placeholder with the value.

```
const dataObj = JSON.parse(data)
```

```
const cardsHtml = dataObj  
  .map((el) => replaceTemplate(tempCard, el))  
  .join("");
```

- ❑ This will return an array of cards template.
- ❑ Now define the `replaceTemplate` method inside your node code.

Filling HTML Template with Dynamic Content



- The `replaceTemplate` method will look like.

```
const replaceTemplate = (temp, product) => {
  let output = temp.replace(/{%PRODUCTNAME%}/g,
    product.productName);
  output = output.replace(/{%FROM%}/g, product.from);
  output = output.replace(/{%NUTRIENTS%}/g,
    product.nutrients);
  output = output.replace(/{%QUANTITY%}/g,
    product.quantity);
  output = output.replace(/{%PRICE%}/g,
    product.price);
  output = output.replace(/{%ID%}/g, product.id);
  output = output.replace(/{%IMAGE%}/g,
    product.image);
  output = output.replace(/{%DESCRIPTION%}/g,
    product.description);
  if (!product.organic) {
    output = output.replace(/{%NOT_ORGANIC%}/g,
      "not-organic");
  }
  return output;
};
```

Filling HTML Template with Dynamic Content



- Now replace the `{%PRODUCT_CARD%}` with `cardsHtml` string.

```
const output = tempOverview.replace("{%PRODUCT_CARD%", cardsHtml);  
res.end(output);
```

- Now in `createServer` method use `url.parse()` method that will return the object with some properties like `pathname`, `query` etc.

```
const { query, pathname } = url.parse(req.url, true);
```

- This will automatically fetch the ***query and pathname*** from the object and store them in respective variables.

- Now check all the conditions with ***pathname*** variable.

- Now identify the product using `const product = dataObj[query.id]`

- Replace the `tempProduct` template with product. `const output = replaceTemplate(tempProduct, product);`

Filling HTML Template with Dynamic Content



- Also change the link of back button `<a href="/overview" class="product__back">`
- The final code will look like. [FINAL CODE](#)

Creating and Using own Modules



- ❑ If you want to use your code multiple files then you can create the module and put that code inside the module.
- ❑ To create a module first create a folder with any name inside any directory. (***For example folder name is modules***)
- ❑ Create a file inside this folder and put the code that you want to reuse. Assign the code to `module.exports`
- ❑ Now you can simply use the code inside any file by simply importing the module. (***require()***)
- ❑ Code After Module